SERVERLESS ON KUBERNETES

Knative

Scale-to-zero HTTP services & event-driven workloads —
a live PoC on a local `kind` cluster

5-minute walkthrough + live demo

THE IDEA

What is Knative?

- An open-source **Kubernetes add-on** (CNCF) that brings a **serverless** developer experience to any K8s cluster.
- Originally from Google (2018); now a **graduated CNCF project**, vendor-neutral.
- Two independent components:

① Serving

- **Request-driven autoscaling**, including **scale-to-zero**, for HTTP workloads. One YAML replaces Deployment + Service + Ingress + HPA.

② Eventing

- A **pub/sub** layer — Brokers, Triggers, Sources — all speaking the **CloudEvents** standard. Loosely-coupled, event-driven apps.

It runs your normal containers — no proprietary runtime, no code lock-in.

HOW IT WORKS · PART 1

Serving — one object, zero idle cost

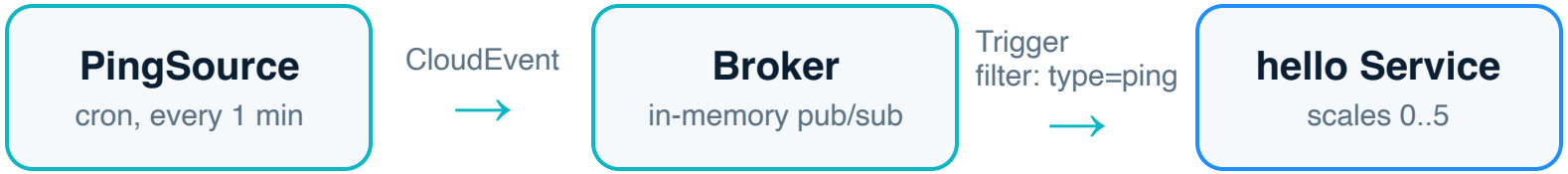
- You apply a single **Knative `Service`**. Knative generates the Deployment, Pod, K8s Service, ingress route & autoscaler for you.
- The **Autoscaler (KPA)** watches **concurrent requests**, not just CPU — scales **0 → N** and back to **0**.
- No traffic ⇒ **no pods ⇒ no cost.** First request triggers a **cold start**; the request waits and is then served.
- Each deploy is an immutable **Revision** → instant rollback & traffic splitting.

```yaml
# manifests/service.yaml
kind: Service        # serving.knative.dev
metadata:
  name: hello
spec:
  template:
    metadata:
      annotations:
        min-scale: "0"   # scale to zero
        max-scale: "5"
        target:    "10"  # req/pod
    spec:
      containers:
        - image: knative-poc-hello:dev
```

HOW IT WORKS · PART 2

Eventing — pub/sub with CloudEvents

- **Source** produces events (timer, Kafka, GitHub, S3…). Here a **PingSource** fires on a cron schedule.
- **Broker** is the central hub that receives and buffers events.
- **Trigger** is a subscription with a **filter** — "route events of `type=…ping` to the `hello` service."



Loose coupling: the source has no idea who consumes its events — and any HTTP service is automatically an event sink.

LIVE DEMO · WHAT'S IN THIS REPO

The PoC we'll run

- **A tiny Go HTTP server** — replies "`hello from <pod>`", and logs any CloudEvent it receives. Same binary serves both roles.
- Runs on a local **`kind`** cluster — `setup.sh` + `deploy.sh` install Knative & deploy it.
- Handles **SIGTERM** for graceful drain on scale-down.

```
# 1 · Serving: cold start → autoscale
demo.sh cold   # wakes a pod 0→1
demo.sh load   # 30s traffic → 2–5 pods
#   …idle 30s → back to 0

# 2 · Eventing: fire an event
demo.sh send-event
# log: "CloudEvent received
#       type=dev.knative.sources.ping"
# a pod spins up just to handle it
```

Punchline: the same scale-to-zero workload is also an event consumer — zero code changes.

TRADE-OFFS

Pros & Cons

Pros

- ✔ **Scale-to-zero** — pay/run nothing when idle.
- ✔ **Less boilerplate** — one YAML vs. 4–5 K8s objects.
- ✔ **No lock-in** — plain containers, runs on any K8s, any cloud.
- ✔ **CloudEvents-native** eventing, pluggable sources.
- ✔ Built-in **revisions, rollbacks, traffic splitting** (canary/blue-green).

Cons

- ✘ **Cold starts** — first request after idle has latency.
- ✘ **You still run Kubernetes** — not zero-ops like a managed FaaS.
- ✘ **Operational complexity** — extra control plane, networking layer (Kourier/Istio).
- ✘ **Mostly HTTP/event-driven** — not ideal for long-lived stateful or batch jobs.
- ✘ Learning curve on top of K8s.

LANDSCAPE

How it compares

TECHNOLOGY	SCALE-TO-ZERO	RUNS ON	LOCK-IN	BEST FOR
Knative	**Yes**	Any Kubernetes	None (containers)	Portable serverless + eventing on K8s
Plain K8s + HPA	No (min 1 pod)	Any Kubernetes	None	Steady-state services
KEDA	Yes (event-based)	Any Kubernetes	None	Event/queue-driven autoscaling
AWS Lambda	Yes	AWS only	High	Pure FaaS, no cluster to run
Google Cloud Run	Yes	GCP (Knative API!)	Medium	Managed Knative, no ops
OpenFaaS	Yes	Any Kubernetes	Low	Function-first simplicity

Knative = the open standard. Cloud Run is literally Google's hosted Knative API — write once, run managed or self-hosted.

TAKEAWAYS

Wrap-up

- **Serving** shrinks an HTTP service's deployment surface to **a single YAML**, with **scale-to-zero** for free.
- **Eventing** gives simple, composable, CloudEvents-native pub/sub — **any HTTP service becomes an event sink**.
- **Portable serverless**: the productivity of Lambda/Cloud Run without the cloud lock-in.
- Trade cold-start latency & some ops overhead for **cost savings and developer velocity**.

Thank you — questions?

SPEAKER SCRIPT · total ≈ 5:00

What to say — slide by slide

Tip: keep four terminals open if running live — `T0` port-forward (`demo.sh pf`), `T1` commands, `T2` pod watch (`demo.sh pods`), `T3` logs (`demo.sh logs`). If you can't run it live, just narrate from the demo slide.

▸ SLIDE 1–2 · 0:00–0:45 · What is Knative?

"Knative is an open-source add-on that turns any Kubernetes cluster into a serverless platform. It has two parts. **Serving** gives request-driven autoscaling — including scale-to-zero — for HTTP workloads. **Eventing** gives a pub/sub layer with brokers, triggers and sources, all speaking the CloudEvents standard. It started at Google, it's now a graduated CNCF project, and importantly it runs your normal containers — no proprietary runtime. I'll show both in about four minutes."

▸ SLIDE 3 · 0:45–1:45 · Serving

Point at the YAML on the right.

"With Serving you apply **one object** — a Knative Service. From this single YAML, Knative creates the Deployment, the pod, the Kubernetes Service, the ingress route and the autoscaler. The key annotations are here: `min-scale 0` means scale to zero when idle, `max-scale 5`, and `target 10` — it autoscales on **concurrent requests per pod**, not CPU. No traffic means no pods, which means no cost. The trade-off is that the first request after idle pays a cold start."

▸ If live: `kubectl get pods -l serving.knative.dev/service=hello` → likely empty (scaled to zero).

SLIDE 3 (continued) · 1:45–2:45 · Cold start + autoscale (live)

- Run `demo.sh cold` — first request wakes a pod, second is instant. Point at T2: a pod appeared.
- Run `demo.sh load` — 30s of traffic. Point at T2: replicas climb to 2–5, then drop back to 0 after ~30s idle.

"That's `min-scale 0`, `max-scale 5` and `target 10` doing their job — no HPA config, no Deployment, no manual scaling. It scaled up under load and back to zero on its own."

SLIDE 4 · 2:45–3:45 · Eventing

"Now Eventing. The **Broker** is a pub/sub hub. The **PingSource** emits a CloudEvent on a cron schedule. The **Trigger** is a subscription with a filter: 'send events of type ping to the hello service.' This is loose coupling — the source has no idea who consumes its events. You can add or remove subscribers without touching the producer."

- If live: `kubectl get broker,trigger,pingsource` to show the wiring.

SLIDE 5 · 3:45–4:30 · Demo / events flowing

- Run `demo.sh send-event` (or wait for the cron tick). Watch the logs in T3.

"There it is — `CloudEvent received type=dev.knative.sources.ping`. And in the pod watch, a pod spun up **just** to handle that event, then scales back down. Here's the punchline: the exact same scale-to-zero HTTP workload is now also an event consumer — with **zero code changes**. It's one Go binary; it just inspects the CloudEvent headers."

SLIDE 6–7 · 4:30–4:50 · Pros/Cons & comparison

"Quick trade-offs: you get scale-to-zero, far less boilerplate, no cloud lock-in, and built-in revisions and traffic splitting. The costs are cold starts, and you still operate Kubernetes plus an extra control plane — it's not zero-ops like Lambda. Compared to alternatives: plain K8s with HPA never scales to zero; KEDA does event-based scaling but no HTTP

serving; Lambda gives you FaaS but locks you to AWS. Knative is the open standard — in fact Google Cloud Run **is** hosted Knative, so you can write once and run it managed or self-hosted."

▸ SLIDE 8 · 4:50–5:00 · Wrap-up

"Two takeaways. One: Knative reduces an HTTP service to a single YAML and gives you scale-to-zero out of the box. Two: the Eventing primitives — Broker, Trigger, Source — are simple, composable and CloudEvents-native, so any HTTP service is automatically an event sink. Portable serverless, without the lock-in. Thank you — happy to take questions."